

Undo Logging:

- Forma de los Update Record: $\langle T_i, X, v \rangle$
 - La transacción T_i actualizó el valor del registro X que **antes** valía v
- Reglas:
 - Cada Update Record $\langle T_i, X, v \rangle$ se baja a disco **antes** de bajar a disco el nuevo valor de X
 - Cada Commit Record $\langle \text{COMMIT } T_i \rangle$ se baja a disco **después** de bajar a disco todos los cambios realizados por T_i
- Una vez que los cambios estén asegurados en el disco, registrar en el log que esos cambios son válidos (commit)
 - Puede haber cambios en el disco que aún no haya registrado como válidos (commit) en el log

Redo logging:

- Forma de los Update Record: $\langle T_i, X, v \rangle$
 - La transacción T_i actualizó el valor del registro X que **pasa a valer** v
- Reglas:
 - Cada Update Record $\langle T_i, X, v \rangle$ se baja a disco **antes** de bajar a disco el nuevo valor de X
 - Cada Commit Record $\langle \text{COMMIT } T_i \rangle$ se baja a disco **antes** de bajar a disco cualquiera de los cambios realizados por T_i
- Primero asegurar los cambios en el log (commit), luego escribirlos en el disco
 - Las actualizaciones en el log no necesariamente quedaron reflejadas en el disco

Undo/Redo logging:

- Forma de los Update Record: $\langle T_i, X, v_0, v_1 \rangle$
 - La transacción T_i actualizó el valor del registro X que **antes** valía v_0 y **pasa a valer** v_1
- Reglas:
 - Cada Update Record $\langle T_i, X, v_0, v_1 \rangle$ se baja a disco **antes** de bajar a disco el nuevo valor de X
- Cada vez que se va a cambiar algo en disco, primero grabar el registro de dicho cambio en el disco

Checkpoint quiescente UNDO logging:

- Etapas:
 - 1 Dejar de aceptar nuevas transacciones
 - 2 Esperar a que todas las transacciones activas (aquellas con Start Record pero sin Commit ni Abort Record) comiencen o aborden
 - 3 Agregar el registro $\langle \text{CKPT} \rangle$ al log y bajarlo a disco
 - 4 Volver a aceptar nuevas transacciones
- Recuperación: Aplicar UNDO desde el final del log hasta el último checkpoint (el primero encontrado)

CKPT no-quiescente UNDO logging:

CKPT no-quiescente REDO logging:

Etapas:

- 1 Escribir $\langle \text{Start CKPT}(T_1, T_2, \dots, T_k) \rangle$ en el log, y efectuar un flush. $T_1, T_2, \dots, T_k$ son las transacciones **activas** al momento de introducir el *checkpoint*.
- 2 Forzar a disco los ítems que fueron escritos en el pool de buffers por transacciones que hicieron *commit* al momento del $\langle \text{Start CKPT} \rangle$, pero que aún no fueron guardados en disco.
- 3 Escribir $\langle \text{End CKPT} \rangle$ en el log y efectuar un flush del mismo.

Recuperación:

- Identificar el último checkpoint que finalizó correctamente (el último $\langle \text{Start CKPT}(\dots) \rangle$ que tiene un $\langle \text{End CKPT} \rangle$ posterior)
- Si se encuentra un $\langle \text{Start CKPT} \rangle$, (no hay $\langle \text{End CKPT} \rangle$ debido a un crash). Ubicar el $\langle \text{End CKPT} \rangle$ **anterior** y su correspondiente $\langle \text{Start CKPT}(T_1, T_2, \dots, T_k) \rangle$ y rehacer todas las transacciones comiteadas que comenzaron a partir de ahí o están entre las T_i .

- **Idea:** Identificar las transacciones incompletas y deshacer los cambios que realizaron en el disco

- Recorrer el log desde el final hacia atrás
- Para cada transacción T_i , recordar si se encuentra un Commit Record o un Abort Record
- Por cada Update Record $\langle T_i, X, v \rangle$,
 - Si se encontró un Commit Record o un Abort Record para T_i , no hacer nada
 - Si no, T_i es una transacción incompleta y hay que deshacer el cambio: Asignar v a X
- Por cada T_i incompleta agregar un Abort Record al log y bajarlo al disco

- **Idea:** Identificar las transacciones completas y rehacer los cambios que no llegaron a bajarse al disco

- Recorrer el log una primera vez para identificar cuáles son las transacciones completas y cuáles las incompletas
- Recorrer el log una segunda vez, desde el principio hacia adelante
- Por cada Update Record $\langle T_i, X, v \rangle$,
 - Si T_i es incompleta (en la primera pasada no se encontró un Commit Record ni un Abort Record para T_i), no hacer nada
 - Si no, T_i es una transacción completa y hay que actualizar el cambio en el disco: Asignar v a X
- Por cada T_i incompleta agregar un Abort Record al log y bajarlo al disco

- Aplicar **UNDO** a las transacciones incompletas en orden inverso

- Recorrer el log desde el final hacia atrás
- Para cada transacción T_i , recordar si se encuentra un Commit Record o un Abort Record
- Por cada Update Record $\langle T_i, X, v_0, v_1 \rangle$,
 - Si se encontró un Commit Record o un Abort Record para T_i , no hacer nada
 - Si no, T_i es una transacción incompleta y hay que deshacer el cambio: Asignar v a X

- Aplicar **REDO** a las transacciones comiteadas en orden

- Recorrer el log una segunda vez, desde el principio hacia adelante
- Por cada Update Record $\langle T_i, X, v_0, v_1 \rangle$,
 - Si T_i es incompleta (en la primera pasada no se encontró un Commit Record ni un Abort Record para T_i), no hacer nada
 - Si no, T_i es una transacción completa y hay que actualizar el cambio en el disco: Asignar v a X

- Por cada T_i incompleta agregar un Abort Record al log y bajarlo al disco

- Etapas:

- 1 Agregar el registro $\langle \text{START CKPT}(T_1, T_2, \dots, T_k) \rangle$ al log y bajarlo al disco (siendo $T_1, T_2, \dots, T_k$ las transacciones activas en el momento)
- 2 Esperar a que todas las transacciones $T_1, T_2, \dots, T_k$ comiencen o aborden, pero sin restringir que empiecen nuevas transacciones
- 3 Agregar el registro $\langle \text{END CKPT} \rangle$ al log y bajarlo a disco

- Recuperación:

- Aplicar UNDO desde el final del log
- Si se encuentra un $\langle \text{END CKPT} \rangle$, seguir hasta el $\langle \text{START CKPT}(\dots) \rangle$
- Si se encuentra un $\langle \text{START CKPT}(T_1, T_2, \dots, T_k) \rangle$, seguir hasta el $\langle \text{START } T_i \rangle$ más antiguo, con $1 \leq i \leq k$. O sea, las que quedaron incompletas.

CKPT no-quiescente UNDO/REDO logging:

Etapas:

- 1 Escribir $\langle \text{Start CKPT}(T_1, T_2, \dots, T_k) \rangle$ en el log, y efectuar un flush. $T_1, T_2, \dots, T_k$ son las transacciones **activas** al momento de introducir el *checkpoint*.
- 2 Escribir a disco todos los buffers "sucios" o *dirty buffers* al momento del $\langle \text{Start CKPT} \rangle$. Escribir a disco **todos** los buffers sucios, no sólo los que hayan sido modificados por transacciones comiteadas al momento del $\langle \text{Start CKPT} \rangle$.
- 3 Escribir $\langle \text{End CKPT} \rangle$ en el log y efectuar un flush del mismo.

Recuperación:

- Transacciones incompletas: para deshacerlas se debe retroceder hasta el start más antiguo de ellas. Agregar registro $\langle \text{ABORT } T_i \rangle$ al log para cada transacción T_i incompleta, y hacer flush del log
- Transacciones con *commit*: Si leyendo desde el último registro se encuentra un $\langle \text{End CKPT} \rangle$ sólo será necesario rehacer las acciones efectuadas desde el correspondiente registro $\langle \text{Start CKPT}(T_1, T_2, \dots, T_k) \rangle$ en adelante. Si no encontramos el $\langle \text{End CKPT} \rangle$, usamos la estrategia del REDO

Concurrencia: Paradigma pesimista

Operaciones conflictivas: mismo ítem y una es escritura

Historias conflicto equivalentes: mismo conj de txs y orden de operaciones conflictivas no abortadas es el mismo

Historia conflicto serializable (SR): conflicto equivalente a historia serial

Grafo de precedencia SG(H): tx nodos, ejes si $T_i < T_j$ y son operaciones conflictivas.

Teorema de serialidad: H es SR sii SG(H) es acíclico.

Historia RC si siempre que T_i lee de T_j , $c_j < c_i$.

Historia ACA si siempre que T_i lee de $T_j \Rightarrow c_j < r_i(X)$

Historia ST si siempre que $w_j(X) < o_i(X) \Rightarrow a_j/c_j < o_i(X)$

Lock binario: $li(X)$, $ui(X)$

Grafo precedencia lock binario: tx nodos, eje $T_i \rightarrow T_j$ si $li(X)$ y luego $lj(X)$.

Lock ternario sin update: $rli(X)$, $wli(X)$

Grafo precedencia ternario: nodos por cada tx, y ejes en orden solo para operaciones conflictivas

Lock Ternario con update: rli , wli , uli

Transacción 2PL: toma locks, libera locks. Una historia es 2PL si cada una de sus tx es 2PL $\Rightarrow$ es SR.

2PL Estricto: 2PL + no libera ningún lock de escritura hasta después del commit. Garantiza historia ST.

2PL Riguroso: 2PL + no libera ningún lock (r/w) hasta desp commit. orden SR = orden de commits.

Detección Deadlock Wait-for Graph: nodo x Tx, eje entre T_i y T_j si T_j espera que T_i libere lock sobre X.

Prevención Deadlock: T_i quiere lock, T_j tiene lock.

Wait-Die: $TS(T_i) < TS(T_j) \Rightarrow T_i$ espera, cc T_i dies. Wound-wait $TS(T_i) < TS(T_j)$, T_j dies. Cc T_i waits.

Optimista:

C/elemento X: $RT(X)$, $WT(X)$, $C(X)$ bool.

Físicamente irrealizable: write-too-late ($TS(T) < RT(X) \Rightarrow$ abort), read-too-late ($TS(T) < WT(X) \Rightarrow$ abort).

Thomas write Rule: si $TS(T) < WT(X)$, skip write.

Dirty data/read: t_i lee X modificado por t_j , pero t_j no commiteo. Se recomienda retrasar la lectura.

Multiversion conflictos: r/w, no mas w/w (porque w genera nueva versión)

Grafo de conflicto multiversion: tx nodos, ejes t_i a t_k si $r_i(X_j)$ y $w_k(X_k)$ tal que $r_i(X_j) < w_k(x_k)$ (precede).

Historia es MCSR sii grafo de conflicto es acíclico.

Protocolo multiversion MVTO:

Cada versión de un elemento de datos lleva un timestamp $ts(t_i)$ de la transacción t_i que fue la que creo la versión.

- Una operación $r_i(x)$ se transforma en una operación multiversion $r_i(w_k)$ donde w_k es la versión de x que tiene el timestamp mas grande menor o igual que $ts(t_i)$ y que fue escrita por t_k , $k \neq i$
- Una operación $w_i(x)$ se procesa de la siguiente manera
 - Si una operación $r_j(x_k)$ tal que $ts(t_k) < ts(t_i) < ts(t_j)$ ya existe en el schedule entonces $w_i(x)$ es rechazada y t_i es abortada. Se produce un *write too late*.
 - en otro caso $w_i(x)$ se transforma en $w_i(x_i)$ y es ejecutada.
- Un commit c_i se retrasa hasta que los commit c_j de todas las transacciones t_j que han escrito nuevas versiones de los elementos de datos leídos por t_i hayan sido ejecutados.

2V2PL:

El protocolo 2V2PL (bloqueo de dos versiones) mantiene como máximo dos versiones de cualquier elemento de datos en cada momento.

- Supongamos que t_i escribe el elemento de datos x , pero aún no está *comiteado*, las dos versiones de x son su imagen anterior y su imagen posterior.
- Tan pronto como t_i se comitea, la imagen anterior puede ser eliminada ya que la nueva versión de x ahora es estable, y las versiones antiguas ya no son necesarias ni se mantienen.

En 2V2PL las operaciones de lectura están restringidas a leer solo las versiones actuales, es decir, la última versión comiteada

	rl	wl
rl	Si	No
wl	No	No

	rl	wl	ul
rl	Si	No	No
wl	No	No	No
ul	Si	No	No

Protocolo multiversion MV2PL (no evita deadlock):

Si el paso no es el final dentro de una transacción:

- Una lectura $r(x)$ se ejecuta de inmediato, asignándole la versión actual del elemento de datos solicitado, es decir, la versión commiteada más recientemente (pero no cualquier otra, previamente commiteada), o asignándole una versión no commiteada de x
- Un escritura $w(x)$ se ejecuta solamente cuando la transacción que ha escrito x por última vez finalizo, es decir no hay otra versión no commiteada de x . O sea se libero el lock de escritura sobre x

Si es el paso final de la transacción t_i , esta se retrasa hasta que commitean las siguientes transacciones

- todas aquellas transacciones t_j que hayan leído la versión actual de un elemento de datos escrito por t_i
- Todas aquellas t_j de las que t_i ha leído algún elemento.

MV2PL puede leer no commiteadas

- rl read lock:** que se establece antes de una operación de lectura $r(x)$ respecto de la versión actual de x
- wl write lock;** que se establece antes de una operación de escritura $w(x)$ para escribir una versión no commiteada de x .
- cl commit o certify lock:** se establece un $cl(x)$ antes de la ejecución del paso final de una transacción en cada elemento de datos x que esta transacción ha escrito .

Las operaciones de unlock deben obedecer al protocolo 2PL. La matriz de compatibilidad es la siguiente

	$rl(x)$	$wl(x)$	$cl(x)$
$rl(x)$	+	+	-
$wl(x)$	+	-	-
$cl(x)$	-	-	-